Project Proposal: MarketScript

MKTG 490 | Spring 2026

Team Member: Tommy Du

What We Are Building

MarketScript is a web-based tool that takes a plain-English description of a marketing data task and returns a working Python script- along with a section-by-section explanation of exactly what the code is doing. Each part of the output has three layers: the code itself, a plain-English annotation explaining the logic, and a warning flag for any lines that make assumptions about the user's data. At the end, the tool also generates a short narrative summary that the user can read out loud to explain the script in a job interview, even if they did not write a single line themselves.

The tool supports multi-turn refinement, meaning the user can say something like "group by region instead of age" and the script updates accordingly, with the explanations refreshed to match. This creates a conversational loop between the user and the tool, rather than a one-shot output.

Why We Are Building It

Marketing roles increasingly expect some level of technical ability. Employers ask about Python, SQL, and analytics tools in interviews, and candidates who cannot engage with those questions are filtered out early. The problem is not that code generation tools do not exist- tools like Claude Code and GitHub Copilot already produce working scripts on demand. The real barrier is different: marketing students cannot read or trust code they did not write. Without understanding what the script does, they cannot verify it, cannot explain it, and cannot catch errors before running it on real data.

Existing tools are built for developers who already know how to read code. MarketScript is built for the opposite user- someone who knows what they want to accomplish but has no way to evaluate whether the output actually does it. The explanation layer is not just a nice-to-have feature, it is the core value proposition. It closes the gap between generating code and understanding it.

How We Are Building It

The prototype will use the OpenAI API as its core AI component. The system prompt will enforce a structured output schema with four consistent sections: the code block, a plain-English explanation for each part, a list of assumption flags, and a closing interview summary. This reflects the prompt engineering principles from Lecture 2- deliberately designing the input to produce a reliable and consistent output format.

On top of that, the tool will maintain a conversation history across turns so users can refine the output in plain English. This agentic loop- where the model receives both the original task and the full prior conversation as context before generating a revised response- aligns with the tool use and context management concepts covered in Lecture 3.

A stretch goal is to add a code execution tool that runs the generated script and feeds any error messages back into the loop, letting the model self-correct before presenting the final output to the user. This will be pursued if time allows, but the core prototype does not depend on it.

How We Will Evaluate It

Success will be measured along two dimensions. First, functional accuracy: whether the generated scripts run without errors on real marketing datasets, tested across a set of common tasks like grouping survey responses, filtering sales data, and computing summary statistics. Second, explanation quality: whether the plain-English annotations accurately reflect what the corresponding code is doing, evaluated by having classmates- who are also marketing students with no coding background- read the explanations and verify they match the code's behavior.

These two metrics are straightforward to test and directly reflect the tool's core promise: code that works, and explanations that actually help the user understand it.